

In C, when an array is passed to a function, it does not behave like a typical variable passed by value. Instead, the array name decays into a pointer to its first element, meaning the function receives the memory address of the original array rather than a full copy. This creates behavior similar to call by reference, even though C formally uses call by value. In a true call by value scenario (as with simple variables), a copy of the data is passed, so any modifications inside the function do not affect the original variable. However, with arrays, because the function works with the address of the original data, any changes made to the elements inside the function directly modify the original array. This is why arrays appear to be passed by reference. The impact is significant: functions can efficiently manipulate large datasets without copying them, but it also means programmers must be careful, as unintended modifications to the array will persist outside the function.